

评分：_____

南京信息工程大学

《面向对象程序设计实践》课程 综合实训报告

题 目 _____ 模拟教务系统 _____

学 院： _____ 计算机学院 _____

班 级： _____ 计科专业腾讯 1 班 _____

小组编号： _____ 第 2 组 _____

学 号： _____ 202383290121 _____

姓 名： _____ 梅应宝 _____

任课教师： _____ 耿兴华 _____

学 期： _____ 2025-2026 (1) _____

模拟教务系统

Github 仓库: [MEIYBAO/OOD-Project: 面向对象程序设计课程设计](#)

1 系统分析

1.1 应用系统简介

本课设面向高校教务业务场景，设计并实现一套“模拟教务系统”。系统围绕“课程—教师—学生—教学班—成绩”主业务链，支持学生选课、教师授课管理、教学班分配、成绩生成与查询等功能，旨在通过面向对象方法完成业务抽象、模块划分、类设计与数据库落地，并在可运行的软件原型中体现封装、继承、多态等核心思想。

系统在需求层面具有如下约束：

- (1) 业务边界：聚焦教务核心流程，不覆盖财务缴费、宿舍、图书等外围模块；
- (2) 数据一致性：选课记录、教学班分配与成绩结果必须在数据库中可追踪、可回溯；
- (3) 权限约束：学生、教师、教务/管理员角色的操作边界清晰；
- (4) 可扩展性：预留“学业监测/预警”扩展点，便于后期增量开发。

1.2 系统需求（功能）分析

结合模拟教务业务的典型流程，系统需求分为角色需求与公共需求两类：

1) 学生端需求

学生需要完成账号登录、课程信息浏览、按学期检索课程、提交选课/退课请求、查询个人课表、查询成绩与学分完成情况等。选课数据需落库，并与课程、教师、教学班等信息建立关联，保证后续分班与成绩生成的可操作性。

(2) 教师端需求

教师需要完成账号登录、查看本人可授课程/教学任务、选择授课课程并形成教学班、查看教学班学生名单、在教学结束后录入/导入成绩，并支持按教学班导出成绩清单。教师端的数据输出应能与学生端成绩查询闭环联动。

(3) 教务/管理员端需求

教务端需要维护基础数据（学生、教师、课程、专业等）、配置开课计划、组织选课、执行教学班分配（例如依据容量上限或规则分班），并在学期末汇总成绩。为提升系统完整性，可提供基本统计功能（课程选课人数、课程通过率等）以及数据导入导出接口。

(4) 公共需求

系统应提供统一的身份认证与权限控制、关键操作日志（选课、分班、成绩生成/修改）、异常处理与输入校验，并保证数据库操作的原子性（必要时采用事务），以降低数据不一致风

险。

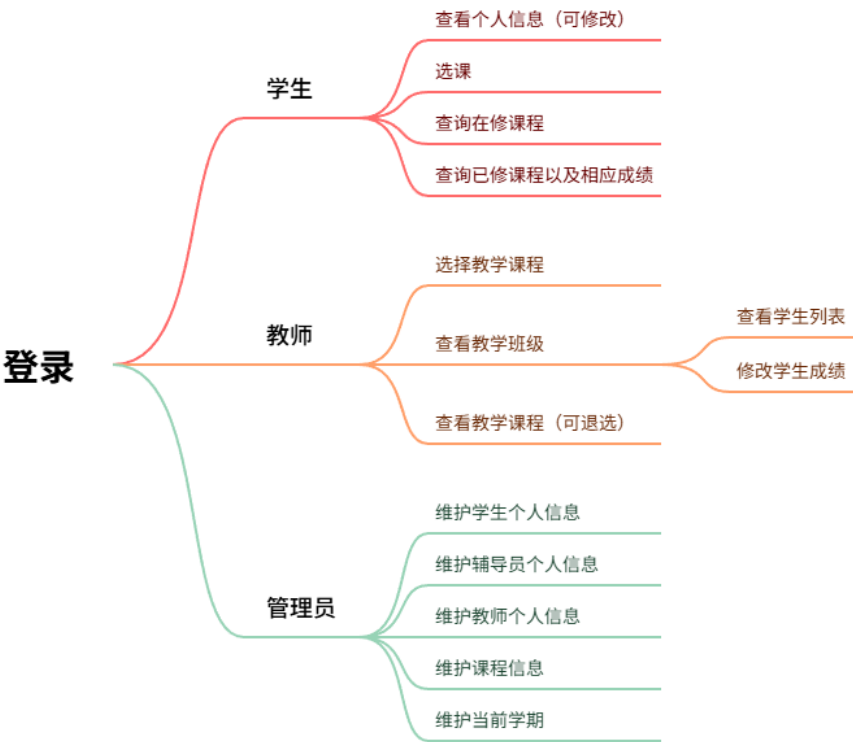
2 系统设计

2.1 总体设计

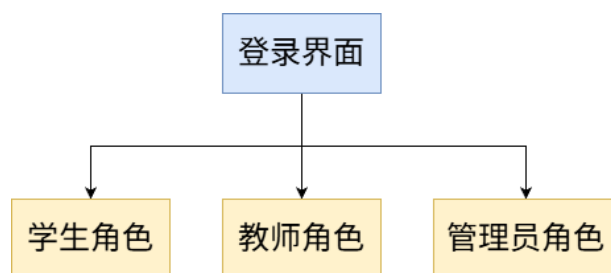
系统采用“分层 + 模块化”的总体设计思路，将复杂业务按职责拆解为若干相对独立的功能单元，并通过统一的数据模型与清晰的接口实现模块协作与解耦。整体架构划分为四层：

1. 表现层（UI/交互）：负责界面展示、输入校验与交互反馈；
2. 业务层（Service/规则）：负责业务流程编排、规则校验与权限控制；
3. 数据访问层（DAO/Repository）：封装数据库读写与查询逻辑；
4. 数据层（MySQL 数据库）：存储基础数据与业务数据，支撑持久化与一致性。

功能模块划分（与界面入口及交互操作一一对应）：

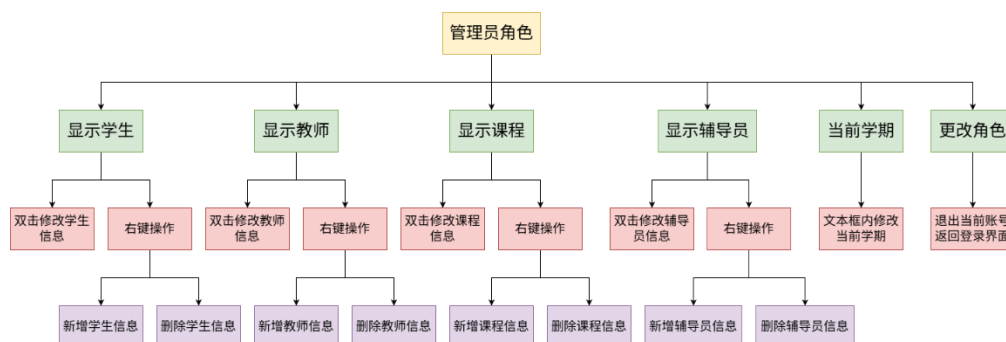


1) 用户与权限模块：提供登录入口与身份选择（学生/教师/管理员）；完成角色识别、权限校验与会话管理；支持切换用户、退出返回登录；管理员端提供角色调整/权限变更入口，确保不同角色仅能访问授权功能。

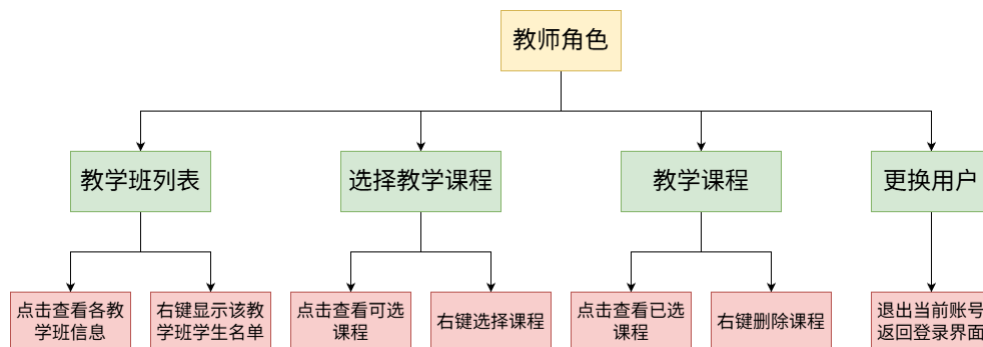


2) 基础信息查询展示模块: 对学生、教师、课程、辅导员等基础信息提供统一的列表展示与明细查看能力，对应界面“显示学生/显示教师/显示课程/显示辅导员”等功能入口，便于各角色快速获取全局数据。

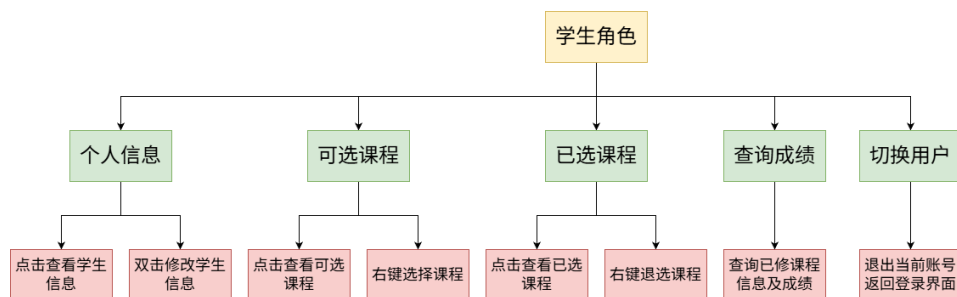
3) 基础数据维护模块（管理员）: 面向管理员提供学生/教师/课程/辅导员数据的新增、删除、修改；支持双击编辑字段与右键菜单快捷增删改；数据变更实时写入数据库并同步刷新界面，保障维护效率与数据一致性。



4) 教学班与授课管理模块（教师）: 支持教师选择授课课程、生成与维护教学班；提供教学班列表查看、按班级查看学生名单等操作；支持已选授课课程的查询与删除，为教务分班、名单维护等流程提供支撑。



5) 选课管理模块（学生）: 支持课程检索与可选课程浏览；提供选课/退课（右键操作）；在业务层完成选课冲突校验与课程容量校验；选课结果持久化到数据库，并与“可选课程/已选课程”视图联动更新。



6) 成绩管理模块: 学生端提供成绩查询（已修课程与成绩展示）；教师/教务端支持成绩录入与批量导入；提供成绩统计分析与导出接口，便于后续扩展报表与数据分析功能。

2.2 详细设计

2.2.1 数据结构设计

本系统采用 MySQL 关系型数据库对教务业务数据进行统一管理。围绕高校教务管理的核心流程，数据库设计覆盖学生、教师、课程、教学班、选课与成绩等关键实体，并通过主外键约束与触发器机制保证数据一致性。

在选课与成绩管理方面，系统设计了 `courseSelection` 表，用于记录学生在不同学期、不同修读状态下的课程选择情况。通过引入 `FirstRepair` 字段，实现了对首次修读与重修课程的区分，使系统能够真实反映学生的学习过程。

在教学组织方面，系统采用 `teacher_course` 与 `teach_class` 两级结构对教师授课行为进行建模。其中，`teacher_course` 表用于描述教师在某一学期承担某门课程的教学任务；`teach_class` 表则作为具体教学班实例，记录课程安排与教学信息。为减少业务层复杂度并避免数据不一致，系统通过数据库触发器在 `teacher_course` 插入时自动生成对应教学班，从而实现教学班的自动化管理。

1) student（学生基础信息表）

主键：**student_uid**（**CHAR(12)**），学号唯一标识。

student_uid	学号 / 唯一 ID（主键，NOT NULL）
name	姓名（VARCHAR(128)）
major	专业（VARCHAR(128)）
telephone	手机号（VARCHAR(32)）
wechat_id	微信号（VARCHAR(64)）
email	邮箱（VARCHAR(255)）

2) teacher（教师基础信息表）

主键：**teacher_uid**（**CHAR(10)**），教师工号唯一标识。

teacher_uid	教师工号 / 唯一 ID（主键，NOT NULL）
-------------	---------------------------

name	姓名 (VARCHAR(128))
telephone	手机号 (VARCHAR(32))
wechat_id	微信号 (VARCHAR(64))
email	邮箱 (VARCHAR(255))

3) counselor (辅导员基础信息表)

主键: **counselor_uid (CHAR(10))**, 辅导员工号唯一标识。

counselor_uid	辅导员工号 / 唯一 ID (主键, NOT NULL)
name	姓名 (VARCHAR(128))
telephone	手机号 (VARCHAR(32))
wechat_id	微信号 (VARCHAR(64))
email	邮箱 (VARCHAR(255))

4) course (课程基础信息表)

主键: **course_uid (CHAR(10))**, 课程编号唯一标识。

course_uid	课程编号 / 唯一 ID (主键, NOT NULL)
course_name	课程名称 (VARCHAR(128))
credits	学分 (INT)
category	课程类别 (VARCHAR(64))

5) user_account (统一登录账号表)

主键: **username (VARCHAR(64))**, 系统登录用户名。

username	登录用户名 (主键, NOT NULL)
password	登录密码 (CHAR(32), MD5 加密存储)
role	用户角色 (VARCHAR(32))
created_at	账号创建时间 (DATETIME, 默认当前时间)

6) teacher_course (教师授课任务表)

主键: **(teacher_uid, course_uid, semester)**, 联合主键。

teacher_uid	教师工号 (外键, 关联 teacher 表)
course_uid	课程编号 (外键, 关联 course 表)
semester	学期 (VARCHAR(16))

7) teach_class (教学班表)

主键: **class_id (CHAR(10))**, 教学班唯一标识。

class_id	教学班编号（主键，NOT NULL）
course_uid	课程编号（外键，关联 course 表）
teacher_uid	教师工号（外键，关联 teacher 表）
semester	学期（VARCHAR(16)）
schedule	上课时间（VARCHAR(64)）
location	上课地点（VARCHAR(64)）

8) courseSelection（选课与成绩过程表）

主键：（student_uid, course_uid, FirstRepair），联合主键。

student_uid	学号（外键，关联 student 表）
course_uid	课程编号（外键，关联 course 表）
selection_date	选课时间（DATETIME）
grade	成绩（DECIMAL(5,2)，可为空）
FirstRepair	首次/重修标识（CHAR(1)，0 为首次，1 为重修）

9) 约束与触发器汇总

一致性约束：全库采用外键将学生/教师/课程等主数据与授课任务、教学班、选课过程强关联，满足“选课—分班—成绩”全链路可追踪目标。

自动化策略：teacher_course 插入后由触发器自动创建 teach_class，避免业务层重复写入与不一致风险。

删除联动策略：对 student/teacher/counselor 的删除使用 BEFORE DELETE 触发器清理从表数据，避免外键冲突并保证账号同步清理。

2.2.2 主要类设计

本系统在实现过程中采用面向对象方法对教务业务进行建模，根据系统功能与代码结构，将主要类划分为**界面控制类（UI/Controller）**、**业务与数据访问类（DAO/Helper）**、**全局配置与工具类**三类。各类之间职责边界清晰，通过接口调用完成协作，从而提高系统的可维护性与扩展性。

（1）界面控制类（UI/Controller 类）

界面控制类主要负责用户交互、界面展示以及对用户操作的响应，其内部通过调用数据访问类完成数据库读写操作，而不直接处理底层数据库细节。

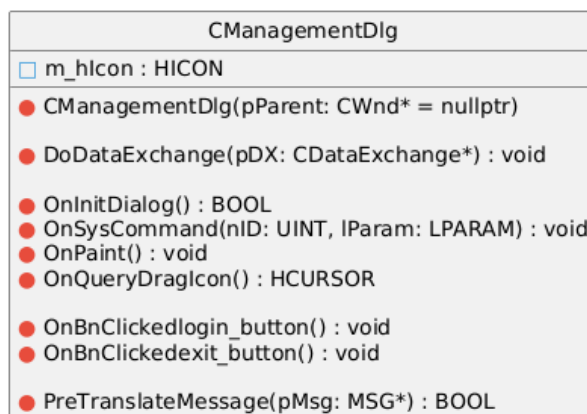
1) CManagementDlg（系统登录与角色分流类）

该类作为系统入口界面，负责完成用户身份认证与角色识别。登录过程中，系统根据用户输入的用户名与密码，查询用户账户表以获取对应角色信息，并根据角色类型打开相应功能界面。该类实现了系统的统一登录控制，是各功能模块的调度中心。

该类主要职责包括：

- 1.接收用户登录信息并完成合法性校验；
- 2.查询用户账户表以确认用户角色；
- 3.根据角色类型分流至学生端、教师端、辅导员端或管理员端界面；
- 4.提供基础的键盘事件处理（如回车触发登录）。

其功能实现与数据库中 `user_account` 表保持一致，通过角色字段区分不同用户权限层级，从而实现系统的权限控制。



2) StudentDlg（学生端功能类）

该类用于实现学生用户的主要业务功能，包括课程浏览、选课与成绩查询。学生在该界面中可查看系统中已开放课程，并根据自身需求进行选课或退课操作。选课结果将被写入数据库选课记录表，供后续教学班分配与成绩管理使用。

该类主要职责包括：

- 1.查询并展示课程信息；
- 2.提交选课请求并写入选课记录；
- 3.提交退选请求并删除相应选课记录；
- 3.查询并展示学生个人已获得成绩信息；
- 4.更新个人信息；
- 5.通过界面事件触发数据库操作。

该类与数据库中的 `courseSelection` 表直接对应，选课操作通过插入选课记录实现，成绩查询则通过读取选课记录中的成绩字段完成。

C StudentDlg
m_list : CListCtrl m_dbHelper : CDatabaseHelper m_currentTable : CString editItem : CEdit hitRow : int hitCol : int m_allowUnselectInCourse : bool
StudentDlg(CWnd* pParent = nullptr) ~StudentDlg() DoDataExchange(CDataExchange* pDX) OnClickedPersonShow() OnInitDialog() : BOOL OnDbcList(NMHDR* pNMHDR, LRESULT* pResult) PreTranslateMessage(MSG* pMsg) : BOOL OnEditKeyDown(UINT nChar, UINT nRepCnt, UINT nFlags) OnEditKillFocus() OnBnClickedChooseClass() OnBnClickedcourses() OnListRClick(NMHDR* pNMHDR, LRESULT* pResult) InsertCourseSelectionAt(CListCtrl& listCtrl, CDatabaseHelper& dbHelper, int nItem, const CString& studentUidParam, const CString& selectionDateParam) : bool DeleteCourseSelectionAt(CListCtrl& listCtrl, CDatabaseHelper& dbHelper, int nItem, const CString& studentUidParam, const CString& selectionDateParam) : bool OnBnClickedStuChange() OnBnClickedSearchGrade()

3) teacherdlg（教师端功能类）

该类用于实现教师端的教学管理功能，主要包括授课课程选择、教学班查看以及学生名单管理等。教师在选择授课课程后，系统会自动生成对应教学班，教师可在界面中查看本人所负责的教学班信息。

类主要职责包括：

- 1.展示教师可授课程列表；
- 2.提交教师授课选择；
- 3.提交取消授课课程；
- 4.查询并展示对应教学班信息；
- 5.查看教学班内学生名单；
- 6.登记教学班内学生成绩。

教师选择授课课程后，系统在数据库中插入授课记录，并通过数据库触发器机制自动生成教学班，从而实现教学组织过程的自动化。该类的业务实现与 teacher_course 表、courseSelection 表及 teach_class 表紧密关联。

C teacherdlg
m_dbHelper : CDatabaseHelper m_list : CListCtrl m_teacherUid : CString m_currentTable : CString
teacherdlg(CWnd* pParent = nullptr) ~teacherdlg() SetTeacherUid(const CString& uid) GetTeacherUid() const : CString GetCurrentTable() const : CString DoDataExchange(CDataExchange* pDX) OnBnClickedshowclass() OnLvniItemchangedworkpanel(NMHDR* pNMHDR, LRESULT* pResult) OnListRClick(NMHDR* pNMHDR, LRESULT* pResult) OnShowClassStudents() OnBnClickedChoosecourse() OnBnClickedclasses() OnBnClickedchangeuser()

4) class_students (教学班学生列表 / 成绩与选课记录编辑类)

该类用于在教师端以“教学班/课程”为入口，展示该课程在指定学期内的选课学生记录，并提供面向表格数据的可视化编辑能力。。

该类主要职责包括：

1.在教师端需要查看教学班学生名单时，提供统一的“学生选课记录列表”展示界面，通过 `course_uid` 过滤选课记录，实现按课程维度的学生汇总展示；

2.支持按学期过滤数据：优先使用外部传入的 `semester`（通常来自教学班记录），若未传入则回退使用系统全局当前学期；并将“YYYY-1 / YYYY-2”学期格式映射为时间区间，对 `selection_date` 进行范围筛选，以保证展示集合与教学班学期一致；

3.通过 `CListCtrl` 的双击事件实现单元格编辑：用户双击某一行/列后在对应位置弹出编辑框，支持直接修改列表中的字段值，提高教师端维护效率；

4.在编辑提交时进行“唯一定位记录”的安全更新：通过读取列表表头得到字段名，并使用复合主键字段（`student_uid + course_uid + FirstRepair`）构造 `WHERE` 条件，确保仅更新唯一一条选课记录，避免误更新整表数据；

5.对输入值进行基础防护处理：例如对单引号进行转义，降低 `SQL` 拼接导致的语句错误风险；同时在缺失关键主键列或主键值不完整时拒绝修改，保证更新操作的可控性与数据一致性。

通过引入该类，系统将“教学班学生数据展示”与“选课记录可视化维护”集中在一个独立界面中实现，使 `teacherdlg` 等业务界面类更专注于流程控制（如选择授课课程、进入教学班视图），而将表格展示、双击编辑、唯一键定位更新等通用交互逻辑封装在专用类中。该设计在工程上提升了教师端的数据可操作性，并且通过复合主键约束与学期口径过滤机制，保证了对 `courseSelection` 数据读写的一致性与可追溯性。

class_students				
m_pDb : CDatabaseHelper* = nullptr m_courseUid : CString m_semester : CString m_studentsList : CListCtrl editItem : CEdit hitRow : int = -1 hitCol : int = -1 <tr><td> class_students(pParent: CWnd* = nullptr) ~class_students() <tr><td> SetDatabaseHelper(pDb: CDatabaseHelper*) : void SetCourseUid(uid: CString&) : void SetSemester(sem: CString&) : void <tr><td> DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr></td></tr></td></tr></td></tr>	class_students(pParent: CWnd* = nullptr) ~class_students() <tr><td> SetDatabaseHelper(pDb: CDatabaseHelper*) : void SetCourseUid(uid: CString&) : void SetSemester(sem: CString&) : void <tr><td> DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr></td></tr></td></tr>	SetDatabaseHelper(pDb: CDatabaseHelper*) : void SetCourseUid(uid: CString&) : void SetSemester(sem: CString&) : void <tr><td> DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr></td></tr>	DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr>	OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void
class_students(pParent: CWnd* = nullptr) ~class_students() <tr><td> SetDatabaseHelper(pDb: CDatabaseHelper*) : void SetCourseUid(uid: CString&) : void SetSemester(sem: CString&) : void <tr><td> DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr></td></tr></td></tr>	SetDatabaseHelper(pDb: CDatabaseHelper*) : void SetCourseUid(uid: CString&) : void SetSemester(sem: CString&) : void <tr><td> DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr></td></tr>	DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr>	OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void	
SetDatabaseHelper(pDb: CDatabaseHelper*) : void SetCourseUid(uid: CString&) : void SetSemester(sem: CString&) : void <tr><td> DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr></td></tr>	DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr>	OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void		
DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr>	OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void			
OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void				

该类主要职责包括：

- 1.建立与维护数据库连接；
- 2.执行查询与更新类 SQL 语句；
- 3.封装通用的数据插入、修改与删除操作；
- 4.将查询结果转换为界面可展示的数据结构。

通过引入该类，系统实现了界面逻辑与数据访问逻辑的分离，提高了代码复用性与系统整体可维护性。

CDatabaseHelper	
m_pConnection : MYSQL* m_bConnected : BOOL	
CDatabaseHelper() ~CDatabaseHelper()	
Connect(strHost: CString& = "localhost", strUser: CString& = "root", strPassword: CString& = mysql_password, strDatabase: CString& = "Schooldb", nPort: int = 3306) : BOOL Disconnect() : void IsConnected() const : BOOL	
DisplayTableData(listCtrl: CListCtrl&, strTableName: CString&, strColumns: CString& = "*", strWhere: CString& = "") : BOOL	
InsertRecord(strTableName: CString&, arrFields: CStringArray&, arrValues: CStringArray&) : BOOL UpdateRecord(strTableName: CString&, strWhere: CString&, arrFields: CStringArray&, arrValues: CStringArray&) : BOOL DeleteRecord(strTableName: CString&, strWhere: CString&) : BOOL	
Utf8ToCString(utf8Str: char*) : CString CStringToUtf8(str: CString&) : CStringA	
ExecuteQuery(strSQL: CString&) : BOOL EscapeString(strValue: CString&) : CString	

2) InputDlg 类（通用输入对话框类）

该类用于封装系统中的通用输入行为，主要解决系统中多个界面在数据维护、信息录入过程中频繁需要弹出输入对话框并获取用户输入的问题。

该类主要职责包括：

- 1.在系统需要用户输入数据时，提供统一的输入对话框界面，通过弹出方式获取用户输入内容，避免在不同业务界面中重复设计输入控件；
- 2.利用 MFC 的 DDX 数据交换机制，将输入控件内容与成员变量进行绑定，实现界面数据与程序数据之间的自动同步；
- 3.通过可配置的提示信息机制（Hint），在对话框中动态显示当前输入字段的说明，提高用户输入的准确性与友好性；
- 4.在用户确认输入时，统一获取并返回输入结果，由调用界面类进一步触发相应的业务逻辑或数据库更新操作；
- 5.配合界面控制类实现数据维护与编辑操作，使业务界面类专注于流程控制而非输入细节处理。

通过引入该类，系统将“用户输入交互”从 StudentDlg、teacherdlg、manager 等具体业务界面类中抽离出来，实现了输入逻辑的集中管理。该设计有效降低了界面类之间的耦合度，使各业务界面能够更加专注于自身功能实现，体现了面向对象设计中封装性与单一职责原则的思想。

InputDialog	
☐	m_strInput : CString
☐	m_strHint : CString
☒	InputDialog(pParent: CWnd* = nullptr)
☒	~InputDialog()
☒	DoDataExchange(pDX: CDataExchange*) : void
☒	OnBnClickedOk() : void
☒	SetHint(hint: CString&) : void
☒	OnInitDialog() : BOOL

(3) 全局配置与工具类

① Global（全局配置类）

该类用于存储系统运行过程中需要共享的全局变量，如当前学期信息、数据库连接参数等。通过集中管理全局配置，避免了参数在多处重复定义的问题。

② tool（工具类）

该类提供若干通用工具函数，用于界面控件操作、字符串处理等辅助功能，减少了界面控制类中的冗余代码。

(4) 类设计与数据库结构的对应关系说明

系统主要类与数据库表之间保持一一对应或一对多的关系：

- 1.登录与权限控制类对应用户账户表；
- 2.学生与教师功能类对应选课表、授课表及教学班表；
- 3.管理员功能类对应基础信息表；
- 4.数据访问辅助类统一负责所有数据库表的操作。

通过上述设计，系统在类结构层面与数据库结构层面形成了清晰、稳定的映射关系，为系统的功能实现与后期扩展奠定了良好基础。

3 系统实现

3.1 系统开发环境

本系统在 Windows 平台下完成开发与调试，采用 C++ 语言结合图形化界面框架实现教务管理功能，并使用 MySQL 关系型数据库作为后端数据存储。系统整体开发环境配置如下。

在操作系统方面，系统运行于 Windows 桌面环境下，依托 **Visual Studio** 提供的集成开发环境完成程序的编译、调试与运行。开发过程中使用 Visual Studio 的 **MFC (Microsoft Foundation Classes)** 框架构建图形用户界面，通过对话框形式实现登录界面及学生端、教师端、管理员端等功能界面，从而提升系统的交互性与可操作性。

在程序设计语言方面，系统核心逻辑采用 C++ 语言实现。通过面向对象方法对教务业务进行建模，将系统划分为界面控制类、数据访问类以及辅助工具类等模块，各模块之间职责清晰，便于维护与扩展。程序使用 Visual Studio 工程文件（.sln、.vcxproj）进行组织，支持多源文件协同编译。

在数据库方面，系统采用 MySQL 作为后端数据库管理系统，用于存储学生、教师、课程、选课、教学班及用户账户等核心数据。数据库字符集统一设置为 utf8mb4，以保证对中文信息的良好支持。系统通过 MySQL 提供的 C API 与数据库进行通信，实现数据的增删改查操作。数据库结构及示例数据通过统一的初始化脚本完成创建与导入，便于系统部署与测试。

在数据管理与调试方面，开发过程中可借助 MySQL 图形化管理工具对数据库表结构、数据内容及执行结果进行检查与验证，以辅助程序功能调试和测试验证。数据库脚本中预置了多角色用户账号及示例选课、授课与成绩数据，为系统功能测试提供了完整的数据基础。

在版本管理方面，系统源代码通过 Git 进行集中管理，便于团队成员协同开发与版本迭代。通过版本控制工具，能够有效追踪代码修改历史，降低多人协作过程中产生的冲突风险。

Github 仓库：[MEIYBAO/OOD-Project: 面向对象程序设计课程设计](https://github.com/MEIYBAO/OOD-Project)。

3.2 程序编码

从界面结构来看，这个模拟教务系统以“登录→角色入口→角色功能面板”为主线，围绕学生、教师、管理员三类用户提供差异化功能与交互方式。

（1）由于考虑到操作的主要目标是数据库，在实现详细系统功能之前，我们先进行了关于数据库功能的封装，连接数据库，通用查询与结果展示，数据的增删改。

3.2.1 数据库连接：

```
BOOL CDatabaseHelper::Connect(const CString& strHost, const CString& strUser,
    const CString& strPassword, const CString& strDatabase, int nPort)
{
    // 如果已经连接，先断开
    if (m_bConnected)
        Disconnect();

    // 初始化MySQL
    m_pConnection = mysql_init(NULL);
    if (m_pConnection == NULL)
    {
        AfxMessageBox(_T("MySQL 初始化失败！"));
        return FALSE;
    }

    // 转换字符串为ANSI
    CStringA strHostA(strHost);
    CStringA strUserA(strUser);
```

```

CStringA strPasswordA(strPassword);
CStringA strDatabaseA(strDatabase);
// 连接数据库
if (!mysql_real_connect(m_pConnection,
    strHostA,
    strUserA,
    strPasswordA,
    strDatabaseA,
    nPort, NULL, 0))
{
    CString strError;
    strError.Format(_T("数据库连接失败! 错误: %hs"), mysql_error(m_pConnection));
    AfxMessageBox(strError);
    mysql_close(m_pConnection);
    m_pConnection = NULL;
    return FALSE;
}
// 设置字符集为utf8
if (mysql_set_character_set(m_pConnection, "utf8") != 0)
{
    CString strError;
    strError.Format(_T("设置字符集失败! 错误: %hs"), mysql_error(m_pConnection));
    AfxMessageBox(strError);
    mysql_close(m_pConnection);
    m_pConnection = NULL;
    return FALSE;
}
m_bConnected = TRUE;
return TRUE;
}

```

3. 2. 2 通用查询与结果展示:

```

BOOL CDatabaseHelper::DisplayTableData(CListCtrl& listCtrl,
    const CString& strTableName,
    const CString& strColumns,
    const CString& strWhere)
{
    if (!m_bConnected)
    {
        AfxMessageBox(_T("数据库未连接!"));
        return FALSE;
    }
    CWaitCursor wait; // 显示等待光标
    // 清空List Control
    listCtrl.DeleteAllItems();
}

```

```

// 删除所有列（安全检查：仅在控件有效且存在列时删除）
if (listCtrl.GetSafeHwnd())
{
    CHeaderCtrl* pHeader = listCtrl.GetHeaderCtrl();
    if (pHeader != nullptr)
    {
        int nColumnCount = pHeader->GetItemCount();
        for (int i = nColumnCount - 1; i >= 0; --i)
        {
            listCtrl.DeleteColumn(i);
        }
    }
}

// 构建 SQL 查询
CString strSQL;
if (strWhere.IsEmpty())
{
    strSQL.Format(_T("SELECT %s FROM %s"), strColumns, strTableName);
}
else
{
    strSQL.Format(_T("SELECT %s FROM %s WHERE %s"), strColumns, strTableName, strWhere);
}
CStringA strSQLA = CStringToUtf8(strSQL);
// 执行查询
if (mysql_query(m_pConnection, strSQLA))
{
    CString strError;
    strError.Format(_T("查询失败！错误：%hs"), mysql_error(m_pConnection));
    AfxMessageBox(strError);
    return FALSE;
}
MYSQL_RES* res = mysql_store_result(m_pConnection);
if (!res)
{
    CString strError;
    strError.Format(_T("获取结果失败！错误：%hs"), mysql_error(m_pConnection));
    AfxMessageBox(strError);
    return FALSE;
}
int num_fields = mysql_num_fields(res);
MYSQL_FIELD* fields = mysql_fetch_fields(res);
// 设置 List Control 列
for (int i = 0; i < num_fields; ++i)
{

```



```

        CString colName = Utf8ToCString(fields[i].name);
        listCtrl.InsertColumn(i, colName, LVCFMT_LEFT, 120);
    }
    // 设置List Control 扩展样式
    listCtrl.SetExtendedStyle(LVS_EX_FULLROWSELECT | LVS_EX_GRIDLINES | LVS_EX_DOUBLEBUFFER);
    // 填充数据
    int rowIndex = 0;
    MYSQL_ROW row;
    while ((row = mysql_fetch_row(res)) != NULL)
    {
        // 第一列
        CString strValue = Utf8ToCString(row[0]);
        int nItem = listCtrl.InsertItem(rowIndex, strValue);
        // 其余列
        for (int i = 1; i < num_fields; ++i)
        {
            CString strCol;
            if (row[i])
            {
                strCol = Utf8ToCString(row[i]);
            }
            else
            {
                strCol = _T("NULL");
            }
            listCtrl.SetItemText(nItem, i, strCol);
        }
        ++rowIndex;
    }
    // 自动调整列宽
    for (int i = 0; i < num_fields; i++)
    {
        listCtrl.SetColumnWidth(i, LVSCW_AUTOSIZE_USEHEADER);
    }
    // 清理结果集
    mysql_free_result(res);
    return TRUE;
}

```

3.2.3 增删改:

```

BOOL CDatabaseHelper::InsertRecord(const CString& strTableName, const CStringArray& arrFields, const CStringA
rray& arrValues)
{
    if (!m_bConnected || arrFields.GetSize() != arrValues.GetSize())
        return FALSE;
}

```

```

CString strSQL;
strSQL.Format(_T("INSERT INTO %s ("), strTableName);
// 添加字段名
for (int i = 0; i < arrFields.GetSize(); i++)
{
    strSQL += arrFields[i];
    if (i < arrFields.GetSize() - 1)
        strSQL += _T(", ");
}
strSQL += _T(") VALUES (");
// 添加字段值
for (int i = 0; i < arrValues.GetSize(); i++)
{
    strSQL += _T("'") + EscapeString(arrValues[i]) + _T("'");
    if (i < arrValues.GetSize() - 1)
        strSQL += _T(", ");
}
strSQL += _T(")");
return ExecuteQuery(strSQL);
}

BOOL CDatabaseHelper::UpdateRecord(const CString& strTableName, const CString& strWhere, const CStringArray&
arrFields, const CStringArray& arrValues)
{
    if (!m_bConnected || arrFields.GetSize() != arrValues.GetSize())
        return FALSE;
    CString strSQL;
    strSQL.Format(_T("UPDATE %s SET "), strTableName);
    // 添加更新字段
    for (int i = 0; i < arrFields.GetSize(); i++)
    {
        strSQL += arrFields[i] + _T(" = ") + EscapeString(arrValues[i]) + _T("'");
        if (i < arrFields.GetSize() - 1)
            strSQL += _T(", ");
    }
    if (!strWhere.IsEmpty())
    {
        strSQL += _T(" WHERE ") + strWhere;
    }
    return ExecuteQuery(strSQL);
}

BOOL CDatabaseHelper::DeleteRecord(const CString& strTableName, const CString& strWhere)
{
    if (!m_bConnected)
        return FALSE;
    CString strSQL;

```

```

        if (strWhere.IsEmpty())
        {
            strSQL.Format(_T("DELETE FROM %s"), strTableName);
        }
        else
        {
            strSQL.Format(_T("DELETE FROM %s WHERE %s"), strTableName, strWhere);
        }
        return ExecuteQuery(strSQL);
    }
}

```

(2) 登录与角色切换

3.2.4 系统提供统一的**登录界面**，用户登录账号后，系统根据用户身份进入不同界面。

```

void CManagementDlg::OnBnClickedlogin_button()
{
    // TODO: 在此添加控件通知处理程序代码

    MYSQL* conn = mysql_init(NULL);

    if (conn == NULL) {
        AfxMessageBox(_T("MySQL 初始化失败!"));
        return;
    }

    // 连接数据库
    if (!mysql_real_connect(conn, "localhost", "root", mysql_password.c_str(), "Schooldb", 3306, NULL, 0)) {
        AfxMessageBox(_T("数据库连接失败!"));
        mysql_close(conn);
        return;
    }

    // 获取用户输入的用户名和密码
    CString strusername;
    GetDlgItemText(username_input, strusername);
    CString strpassword;
    GetDlgItemText(password_input, strpassword);

    // 构造 SQL 语句
    CStringA usernameA(strusername);
    CStringA passwordA(strpassword);
    CStringA sqlA;
    sqlA.Format("SELECT role FROM user_account WHERE username='%s' AND password=MD5('%s')",
        usernameA.GetString(), passwordA.GetString());
    uid = usernameA.GetString();
    if (mysql_query(conn, sqlA.GetString()) == 0) {
        MYSQL_RES* res = mysql_store_result(conn);
        if (res && mysql_num_rows(res) > 0) {
            MYSQL_ROW row = mysql_fetch_row(res);
            CStringA roleA(row[0] ? row[0] : "");
            CString role(roleA);

```

```

AfxMessageBox(_T("登录成功! "));

mysql_free_result(res);

mysql_close(conn);

EndDialog(IDOK);

if (role == _T("student")) {

    // 学生界面

    StudentDlg dlg;

    dlg.DoModal();

}

else if (role == _T("teacher")) {

    // 教师界面

    // TeacherDlg dlg;

    // dlg.DoModal();

    teacherdlg teacher; // 示例用 Testdlg

    teacher.SetTeacherUid(strusername);

    teacher.DoModal();

}

else if (role == _T("counselor")) {

    // 辅导员界面

    // CounselorDlg dlg;

    // dlg.DoModal();

    Testdlg dlg; // 示例用 Testdlg

    dlg.DoModal();

}

else if (role == _T("manager")) {

    manager manager_dlg;

    manager_dlg.DoModal();

}

else {

    AfxMessageBox(_T("未知身份, 无法进入系统! "));

}

return;

}

else {

    AfxMessageBox(_T("用户名或密码错误! "));

    if (res) mysql_free_result(res);

}

}

else {

    AfxMessageBox(_T("查询失败! "));

}

mysql_close(conn);

}

```

3. 2. 5 各角色界面均支持退出当前账号并返回登录界面，实现会话切换与重新登录。

```

void manager::OnBnClickedchangeid()
{
    // TODO: 在此添加控件通知处理程序代码
    CManagementDlg dlg;
    EndDialog(IDOK);
    dlg.DoModal();
}

```

(3)管理员端功能（数据维护与教务参数）

管理员界面以“展示 + 维护”为核心，覆盖基础数据的全量管理：

3.2.6 基础数据展示：支持一键显示学生信息、教师信息、课程信息、辅导员信息（对应“显示学生/显示教师/显示课程/显示辅导员”。

展示函数封装在 CDatabaseHelper 类中，调用 3.2.2 中的如下函数即可实现

```

BOOL CDatabaseHelper::DisplayTableData(CListCtrl& listCtrl,
    const CString& strTableName,
    const CString& strColumns,
    const CString& strWhere);

```

3.2.7 数据编辑交互：支持通过双击修改学生/教师/课程/辅导员信息；

```

void manager::OnDblclkList(NMHDR* pNMHDR, LRESULT* pResult)
{
    LPNMITEMACTIVATE pNMItemActivate = reinterpret_cast<LPNMITEMACTIVATE>(pNMHDR);
    *pResult = 0;
    LVHITTESTINFO info;
    info.pt = pNMItemActivate->ptAction;
    if (m_list.SubItemHitTest(&info) != -1)
    {
        hitRow = info.iItem;
        hitCol = info.iSubItem;
        if (editItem.m_hWnd == NULL) {
            CRect dummyRect(0, 0, 0, 0);
            editItem.Create(WS_CHILD | ES_LEFT | WS_BORDER | ES_AUTOHSCROLL, dummyRect, this, 101);
            editItem.SetFont(this->GetFont(), FALSE);
        }
        // 获取单元格的精确矩形
        CRect rect;
        m_list.GetSubItemRect(info.iItem, info.iSubItem, LVIR_LABEL, rect);
        // 将ListCtrl 的坐标转换为对话框坐标
        CPoint pt(rect.TopLeft());
        m_list.ClientToScreen(&pt);
        ScreenToClient(&pt);
        rect.MoveToXY(pt);
        // 设置编辑框内容和位置
        editItem.SetWindowText(m_list.GetItemText(info.iItem, info.iSubItem));
    }
}

```

```

        editItem.MoveWindow(&rect, TRUE);

        editItem.ShowWindow(SW_SHOW);

        editItem.SetFocus();

        editItem.SetSel(0, -1); // 全选文本，方便直接编辑
    }
}

```

3.2.8 右键进行快捷操作（常用于增加/删除等菜单式展示）。

```

void manager::OnListRClick(NMHDR* pNMHDR, LRESULT* pResult)
{
    // 判断List Control 是否有数据
    if (m_currentTable == _T("无"))
    {
        *pResult = 0;
        return;
    }
    CMenu menu;
    menu.CreatePopupMenu();
    menu.AppendMenu(MF_STRING, 1001, _T("新增"));
    menu.AppendMenu(MF_STRING, 1002, _T("删除"));
    CPoint pt;
    GetCursorPos(&pt);
    int nCmd = menu.TrackPopupMenu(TPM_RETURNCMD | TPM_LEFTALIGN, pt.x, pt.y, this);
    switch (nCmd)
    {
        case 1001:
            OnMenuAdd();
            break;
        case 1002:
            OnMenuDelete();
            break;
        default:
            break;
    }
    *pResult = 0;
}

```

3.2.9 数据新增：通过点击右键菜单提供的明确按钮入口新增学生/教师/课程/辅导员信息（调用 CDatabaseHelper 类方法）；

```

void manager::OnMenuAdd()
{
    // TODO: 弹出新增对话框，插入数据库并刷新列表
    CString hint;
}

```

```

int colCount = m_list.GetHeaderCtrl()->GetItemCount();
for (int i = 0; i < colCount; ++i)
{
    CString colName = GetListCtrlColumnName(m_list, i);
    hint += colName;

    if (i != colCount - 1) hint += _T(" ");
}
InputDialog dlg(this);
dlg.SetHint(_T("请输入以下字段（用空格分隔）：") + hint);
if (dlg.DoModal() == IDOK)
{
    CString input = dlg.m_strInput; // 用户输入的内容
    if (input.IsEmpty())
        return;

    // 按空格分割
    CStringArray arr;
    int cur = 0;
    while (true)
    {
        CString token = input.Tokenize(_T(" "), cur);
        if (token.IsEmpty() && cur == -1) break;
        if (!token.IsEmpty()) arr.Add(token);
    }

    int colCount = m_list.GetHeaderCtrl()->GetItemCount();
    if (arr.GetSize() < colCount)
    {
        AfxMessageBox(_T("输入项不足！"));
        return;
    }

    // 插入到List Control
    int nItem = m_list.InsertItem(m_list.GetItemCount(), arr[0]);
    for (int i = 1; i < colCount; ++i)
    {
        m_list.SetItemText(nItem, i, arr[i]);
    }

    // 插入数据库
    CStringArray fields, values;
    for (int i = 0; i < colCount; ++i)
    {
        CString cloumnname = GetListCtrlColumnName(m_list, i);
        fields.Add(cloumnname);
        values.Add(arr[i]);
    }

    if (m_dbHelper.InsertRecord(m_currentTable, fields, values))
    {

```

```

        AfxMessageBox(_T("新增成功!"));
    }
    else
    {
        AfxMessageBox(_T("数据库插入失败!"));
        m_list.DeleteItem(nItem);
    }
}
}
}

```

3.2.10 删除学生/教师/课程/辅导员信息（调用 CDatabaseHelper 类方法）：

```

void manager::OnMenuDelete()
{
    // TODO: 获取选中项，删除数据库记录并刷新列表
    // 获取选中项
    POSITION pos = m_list.GetFirstSelectedItemPosition();
    if (pos == nullptr)
    {
        AfxMessageBox(_T("请先选择要删除的记录!"));
        return;
    }
    int nItem = m_list.GetNextSelectedItem(pos);
    CString strID = m_list.GetItemText(nItem, 0); // 假设主键在第0列
    int colIndex = 0; // 你要获取的列索引
    CString colName = GetListCtrlColumnName(m_list, colIndex);
    // 获取当前表名（可根据你的实际逻辑调整）
    CString tableName = m_currentTable;
    // 确认删除
    if (AfxMessageBox(_T("确定要删除该记录吗?"), MB_YESNO | MB_ICONQUESTION) != IDYES)
        return;
    CString strWhere;
    strWhere.Format(_T("%s = '%s'"), colName, strID);
    // 调用数据库删除
    if (m_dbHelper.DeleteRecord(tableName, strWhere))
    {
        m_list.DeleteItem(nItem); // 从界面移除
        AfxMessageBox(_T("删除成功!"));
    }
    else
    {
        AfxMessageBox(_T("删除失败!"));
    }
}
}

```


3.2.11 双击修改数据

```
void manager::OnDblclkList(NMHDR* pNMHDR, LRESULT* pResult)
{
    LPNMITEMACTIVATE pNMItemActivate = reinterpret_cast<LPNMITEMACTIVATE>(pNMHDR);
    *pResult = 0;
    LVHITTESTINFO info;
    info.pt = pNMItemActivate->ptAction;

    if (m_list.SubItemHitTest(&info) != -1)
    {
        hitRow = info.iItem;
        hitCol = info.iSubItem;
        if (editItem.m_hWnd == NULL) {
            CRect dummyRect(0, 0, 0, 0);
            editItem.Create(WS_CHILD | ES_LEFT | WS_BORDER | ES_AUTOHSCROLL, dummyRect, this, 101);
            editItem.SetFont(this->GetFont(), FALSE);
        }
        // 获取单元格的精确矩形
        CRect rect;
        m_list.GetSubItemRect(info.iItem, info.iSubItem, LVIR_LABEL, rect);

        // 将 ListCtrl 的坐标转换为对话框坐标
        CPoint pt(rect.TopLeft());
        m_list.ClientToScreen(&pt);
        ScreenToClient(&pt);
        rect.MoveToXY(pt);

        // 设置编辑框内容和位置
        editItem.SetWindowText(m_list.GetItemText(info.iItem, info.iSubItem));
        editItem.MoveWindow(&rect, TRUE);
        editItem.ShowWindow(SW_SHOW);
        editItem.SetFocus();
        editItem.SetSel(0, -1); // 全选文本，方便直接编辑
    }
}
```

3.2.12 学期管理：管理员可查看“当前学期”，并通过文本框修改当前学期，用于模拟教务周期切换与数据归档依据。

```
void manager::OnEnChangeSemesterEdit()
{
    if (!m_semesterEdit.GetSafeHwnd()) return;
    CString sem;
    m_semesterEdit.GetWindowText(sem);
    sem.Trim(); // 去掉首尾空格，规范化输入
}
```

```

int year = 0, term = 0;

if (_stscanf_s(sem, _T("%d-%d"), &year, &term) == 2)
{
    if ((term == 1 || term == 2) && year >= 2000 && year <= 2100)
    {
        // 规范化为 "YYYY-T" 格式，避免因为空格或前导零等差异导致判断失败
        CString norm;
        norm.Format(_T("%04d-%d"), year, term);
        CStringA semA(norm);
        std::string newSem = semA.GetString();
        // 仅在确实变化时写回全局变量
        if (newSem != semester_now)
        {
            semester_now = newSem;
            AfxMessageBox(CString(_T("学期已更新为: ")) + norm);
        }
    }
    // 非法格式不更新全局变量
}
}

```

(3) 教师端功能（授课选择与教学班管理）

教师界面围绕“授课课程—教学班—学生名单”的链路组织功能：

3.2.13 教学班列表：查看自己负责的教学班，并可点击查看各教学班信息；
实现逻辑与管理员相应实现逻辑类似。

3.2.14 右键显示该教学班的学生名单（名单管理入口）

```

void teacherdlg::OnListRClick(NMHDR* pNMHDR, LRESULT* pResult)
{
    // 仅在有意义的表类型上弹出对应的右键菜单
    // 支持: teach_class -> "显示学生列表"
    //      course      -> "删除" 或 "选择"

    if (m_currentTable.IsEmpty())
    {
        *pResult = 0;
        return;
    }

    // 获取鼠标位置，转换到 List 客户区并 HitTest 确认是在某一项上右击
    CPoint ptScreen;
    GetCursorPos(&ptScreen);
    CPoint ptClient = ptScreen;
    m_list.ScreenToClient(&ptClient);
    int nItem = m_list.HitTest(ptClient);
    if (nItem == -1)

```

```

{
    // 在空白区域不弹菜单
    *pResult = 0;
    return;
}

CMenu menu;
menu.CreatePopupMenu();
if (m_currentTable.CompareNoCase(_T("teach_class")) == 0)
{
    menu.AppendMenu(MF_STRING, 1001, _T("显示学生列表"));
}
else if (m_currentTable.CompareNoCase(_T("course")) == 0)
{
    // 在课程视图: 由“教学课程”填充时显示“删除”, 由“选择课程”填充时显示“选择”
    if (s_allowDeleteInCourse)
        menu.AppendMenu(MF_STRING, 2001, _T("删除"));
    else
        menu.AppendMenu(MF_STRING, 3001, _T("选择"));
}
else
{
    // 其他表类型暂不显示菜单
    *pResult = 0;
    return;
}

int nCmd = menu.TrackPopupMenu(TPM_RETURNCMD | TPM_LEFTALIGN, ptScreen.x, ptScreen.y, this);
switch (nCmd)
{
case 1001:
    OnShowClassStudents();
    break;
case 2001:
{
    // 使用专门的函数删除 teacher_course 表中的记录
    CString semCs = CString(semester_now.c_str());
    bool ok = DeleteTeacherCourseAt(m_list, m_dbHelper, nItem, m_teacherUid, semCs);
    if (ok)
    {
        // 删除成功后通过“教学课程”按钮刷新视图
        OnBnClickedclasses();
    }
}
break;
case 3001:
{

```

```

// 在“选择课程”视图下，插入 teacher_course 记录
CString semCs = CString(semester_now.c_str());
bool ok = InsertTeacherCourseAt(m_list, m_dbHelper, nItem, m_teacherUid, semCs);
if (ok)
{
    // 插入成功后刷新“选择课程”视图，移除已被选择的课程
    OnBnClickedChoosecourse();
}
}
break;
default:
    break;
}
*pResult = 0;
}

```

3.2.15 展示教学班学生列表：

实现逻辑与管理员相应实现（展示数据）类似；

3.2.16 双击修改成绩：

复用函数 3.2.11 即可，实现逻辑与管理员相应实现类似。

```
void manager::OnDblclkList(NMHDR* pNMHDR, LRESULT* pResult);
```

3.2.17 选择教学课程：

点击查看可选课程；实现逻辑与管理员相应实现类似。

3.2.18 右键选择课程（将课程加入授课列表）

调用 CDatabaseHelper 类成员函数即可。实现逻辑与管理员相应实现类似。

3.2.20 教学课程管理：

点击查看已选（已授）课程；实现逻辑与管理员相应实现类似。

3.2.21 右键删除课程（取消授课/移除课程）

实现逻辑与管理员相应实现类似。

3.2.22 用户切换/退出：支持更换用户并返回登录界面。

```

void manager::OnBnClickedchangeid()
{
    // TODO: 在此添加控件通知处理程序代码

    CManagementDlg dlg;
    EndDialog(IDOK);
}

```

```
dlg.DoModal();
}
```

(4) 学生端功能（个人信息、选课退课、成绩查询）

学生界面突出“个人信息—选课—成绩”三块：

3.2.23 个人信息：点击查看学生信息：

调用 CDatabaseHelper 类成员函数即可。

```
void StudentDlg::OnClickPersonShow()
{
    // TODO: 在此添加控件通知处理程序代码

    CString strWhere;
    strWhere.Format(_T("%s = '%s'"), _T("student_uid"), uid);
    if (!m_dbHelper.DisplayTableData(m_list, _T("student"), _T("*"), strWhere))
    {
        AfxMessageBox(_T("显示学生数据失败!"));
    }
    else
    {
        m_currentTable = _T("student");
    }
}
```

3.2.24 双击修改学生信息（模拟资料维护）

```
void StudentDlg::OnDblClkList(NMHDR* pNMHDR, LRESULT* pResult)
{
    // 仅当前视图为 student（即通过 person_show 按钮加载）时允许左键双击编辑
    if (m_currentTable.CompareNoCase(_T("student")) != 0)
    {
        *pResult = 0;
        return;
    }

    LPNMITEMACTIVATE pNMItemActivate = reinterpret_cast<LPNMITEMACTIVATE>(pNMHDR);
    *pResult = 0;
    LVHITTESTINFO info;
    info.pt = pNMItemActivate->ptAction;
    if (m_list.SubItemHitTest(&info) != -1)
    {
        hitRow = info.iItem;
        hitCol = info.iSubItem;
        if (editItem.m_hWnd == NULL) {
            CRect dummyRect(0, 0, 0, 0);
            editItem.Create(WS_CHILD | ES_LEFT | WS_BORDER | ES_AUTOHSCROLL, dummyRect, this, 101);
            editItem.SetFont(this->GetFont(), FALSE);
        }
    }
}
```

```

    }

    // 获取单元格的精确矩形
    CRect rect;

    m_list.GetSubItemRect(info.iItem, info.iSubItem, LVIR_LABEL, rect);

    // 将 ListCtrl 的坐标转换为对话框坐标
    CPoint pt(rect.TopLeft());
    m_list.ClientToScreen(&pt);
    ScreenToClient(&pt);
    rect.MoveToXY(pt);

    // 设置编辑框内容和位置
    editItem.SetWindowText(m_list.GetItemText(info.iItem, info.iSubItem));
    editItem.MoveWindow(&rect, TRUE);
    editItem.ShowWindow(SW_SHOW);
    editItem.SetFocus();
    editItem.SetSel(0, -1); // 全选文本，方便直接编辑
}
}

```

3. 2. 25 可选课程与选课:

点击查看可选课程；实现逻辑与管理员相应实现类似，但要加学期筛选条件。

```

void StudentDlg::OnBnClickedChooseClass()
{
    // TODO: 在此添加控件通知处理程序代码
    m_allowUnselectInCourse = false;

    // 从全局 std::string 转为 CString 并转义
    CString semCs = CString(semester_now.c_str());
    CString semEsc = EscapeSql(semCs);

    int year = 0;
    int term = 0;

    // 解析 "2024-1" 这种格式
    _stscanf_s(semEsc, _T("%d-%d"), &year, &term);

    CString semDateStart;
    CString semDateEnd;

    if (term == 1)
    {
        // 1 学期 -> 3 月 1 日
        semDateStart.Format(_T("%d-3-1"), year);
        semDateEnd.Format(_T("%d-9-1"), year);
    }

    else if (term == 2)
    {
        // 2 学期 -> 9 月 1 日
        semDateStart.Format(_T("%d-9-1"), year);
        ++year;
    }
}

```

```

        semDateEnd.Format(_T("%d-3-1"), year);
    }
    CString strWhere;
    strWhere.Format(_T("course_uid NOT IN (SELECT course_uid FROM courseselection WHERE student_uid = '%s' AND selection_date >= '%s' AND selection_date < '%s')"), uid, semDateStart, semDateEnd);
    // 如果你的数据库不支持子查询, 也可以改为先查询 teacher_course 再用 NOT IN 列表构造 WHERE
    if (!m_dbHelper.DisplayTableData(m_list, _T("course"), _T("*"), strWhere))
    {
        AfxMessageBox(_T("显示未被选择的课程失败!"));
    }
    else
    {
        // 标记当前 ListCtrl 正在显示 course
        m_currentTable = _T("course");
    }
}

```

3.2.26 右键选择课程完成选课

根据选课当前时间判断学期并生成相应的选课记录:

```
void StudentDlg::OnListRClick(NMHDR* pNMHDR, LRESULT* pResult)
```

```

{
    // 只有当当前表是 course 时显示选择/退选菜单
    if (m_currentTable.CompareNoCase(_T("course")) != 0)
    {
        *pResult = 0;
        return;
    }
    // 获取鼠标位置并命中行
    CPoint ptScreen;
    GetCursorPos(&ptScreen);
    CPoint ptClient = ptScreen;
    m_list.ScreenToClient(&ptClient);
    int nItem = m_list.HitTest(ptClient);
    if (nItem == -1)
    {
        *pResult = 0;
        return;
    }
    CMenu menu;
    menu.CreatePopupMenu();
    // 根据视图状态决定是“选择”还是“退选”
    if (m_allowUnselectInCourse)
        menu.AppendMenu(MF_STRING, 4002, _T("退选"));
    else

```

```

        menu.AppendMenu(MF_STRING, 4001, _T("选择"));
int nCmd = menu.TrackPopupMenu(TPM_RETURNCMD | TPM_LEFTALIGN, ptScreen.x, ptScreen.y, this);
switch (nCmd)
{
case 4001: // 选择 -> 插入 courseselection
{
    // 可以传入学期信息或空（函数内部会使用当前日期）
    CString semDate; // 传空使用当前日期
    bool ok = InsertCourseSelectionAt(m_list, m_dbHelper, nItem, uid, semDate);
    if (ok)
    {
        // 插入成功后刷新当前视图（移除已被选择的课程）
        OnBnClickedChooseClass();
    }
}
break;
case 4002: // 退选 -> 删除 courseselection
{
    CString semDate; // 可传空或具体学期日期
    bool ok = DeleteCourseSelectionAt(m_list, m_dbHelper, nItem, uid, semDate);
    if (ok)
    {
        // 退选成功后刷新当前课程视图
        OnBnClickedcourses();
    }
}
break;
default:
    break;
}
*pResult = 0;
}

```

3. 2. 27 已选课程与退课：

点击查看已选课程；实现逻辑与管理员相应逻辑类似，额外添加学期特判：

```

void StudentDlg::OnBnClickedcourses()
{
    // TODO: 在此添加控件通知处理程序代码
    m_allowUnselectInCourse = true;
    // 从全局 std::string 转为 CString 并转义
    CString semCs = CString(semester_now.c_str());
    CString semEsc = EscapeSql(semCs);
    int year = 0;
    int term = 0;
}

```



```

// 解析 "2024-1" 这种格式
_stscanf_s(semEsc, _T("%d-%d"), &year, &term);
CString semDateStart;
CString semDateEnd;
if (term == 1)
{
    // 1 学期 -> 3 月 1 日
    semDateStart.Format(_T("%d-3-1"), year);
    semDateEnd.Format(_T("%d-9-1"), year);
}
else if (term == 2)
{
    // 2 学期 -> 9 月 1 日
    semDateStart.Format(_T("%d-9-1"), year);
    ++year;
    semDateEnd.Format(_T("%d-3-1"), year);
}
CString strWhere;
strWhere.Format(_T("course_uid IN (SELECT course_uid FROM courseselection WHERE student_uid = '%s' AND se
lection_date >= '%s' AND selection_date < '%s')"), uid, semDateStart, semDateEnd);
// 如果你的数据库不支持子查询, 也可以改为先查询 teacher_course 再用 NOT IN 列表构造 WHERE
if (!m_dbHelper.DisplayTableData(m_list, _T("course"), _T("*"), strWhere))
{
    AfxMessageBox(_T("显示已选择的课程失败!"));
}
else
{
    // 标记当前 ListCtrl 正在显示 course
    m_currentTable = _T("course");
}
}
}

```

3.2.28 右键退选课程完成退课。

实现逻辑与管理员删除逻辑类似。

3.2.29 成绩查询：提供“查询成绩”入口，用于查看已修课程信息及成绩结果（与选课结果关联）。

```

void StudentDlg::OnBnClickedSearchGrade()
{
    // TODO: 在此添加控件通知处理程序代码
    m_allowUnselectInCourse = true;
    // 从全局 std::string 转为 CString 并转义
    CString semCs = CString(semester_now.c_str());
}

```

```

CString semEsc = EscapeSql(semCs);

int year = 0;
int term = 0;
// 解析 "2024-1" 这种格式
_stscanf_s(semEsc, _T("%d-%d"), &year, &term);
CString semDateStart;
if (term == 1)
{
    // 1 学期 -> 3 月 1 日
    semDateStart.Format(_T("%d-3-1"), year);
}
else if (term == 2)
{
    // 2 学期 -> 9 月 1 日
    semDateStart.Format(_T("%d-9-1"), year);
}
CString strTables = _T("course JOIN courseselection ON course.course_uid=courseselection.course_uid");
CString strColumns = _T("course.course_uid,course_name,grade,credits,category,FirstRepair");
CString strWhere;
strWhere.Format(_T("student_uid = '%s' AND selection_date < '%s'"), uid, semDateStart);
// 如果你的数据库不支持子查询，也可以改为先查询 teacher_course 再用 NOT IN 列表构造 WHERE
if (!m_dbHelper.DisplayTableData(m_list, strTables, strColumns, strWhere))
{
    AfxMessageBox(_T("显示课程成绩失败!"));
}
else
{
    // 标记当前 ListCtrl 正在显示 course
    m_currentTable = _T("course,courseselection");
}
}

```

3.2.30 用户切换/退出：退出当前账号返回登录界面。

```

void manager::OnBnClickedchangeid()
{
    // TODO: 在此添加控件通知处理程序代码

    CManagementDlg dlg;
    EndDialog(IDOK);
    dlg.DoModal();
}

```

4 系统测试

本章围绕系统的核心功能模块，对模拟教务系统进行功能测试与流程验证。测试以数据

库初始化脚本为基础，结合系统各角色的实际操作流程，从登录、选课、授课、教学班生成以及成绩管理等方面，对系统的正确性与稳定性进行验证。

4.1 测试环境

系统测试环境与开发环境保持一致，具体配置如下：

- 1.操作系统：Windows 桌面环境
- 2.开发与运行环境：Visual Studio（MFC 框架）
- 3.数据库管理系统：MySQL
- 4.数据库字符集：utf8mb4
- 5.测试数据来源：系统初始化数据库脚本（SchoolDB.sql）

测试前，通过执行数据库初始化脚本（SchoolDB.sql）完成数据库创建、表结构生成及示例数据导入，确保系统在统一、可复现的数据环境下进行测试。

4.2 测试方法与原则

系统测试采用**功能测试与流程测试相结合**的方式，重点验证系统是否能够正确支持教务管理的核心业务流程。测试遵循以下原则：

1. 以角色为主线进行测试

分别以学生、教师、管理员三类角色登录系统，验证各角色功能边界是否清晰，权限控制是否正确。

2. 以业务闭环为测试目标

重点测试“选课—授课—教学班—成绩”这一完整业务链，确保数据在各模块之间能够正确流转。

3. 以数据库结果作为最终判据

所有功能测试均通过界面操作与数据库表中数据变化进行双重验证，保证界面展示结果与数据库存储结果一致。

4.3 用户登录与角色分流测试

4.3.1 测试目的

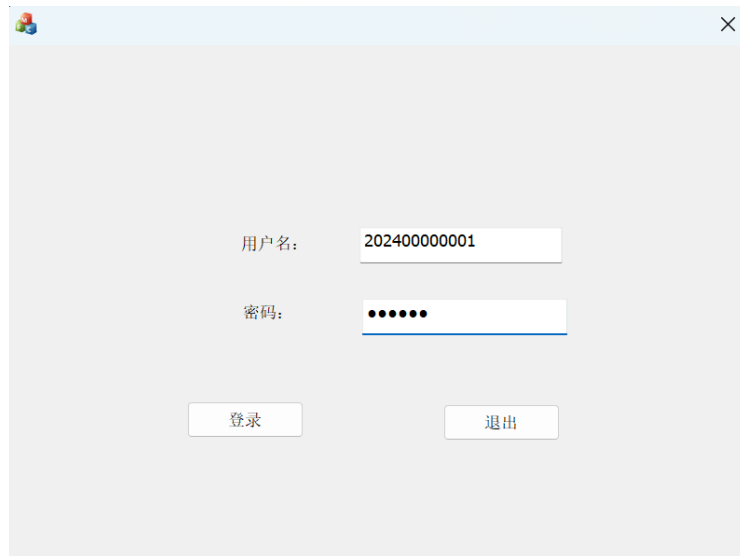
验证系统是否能够正确完成用户身份认证，并根据用户角色进入对应功能界面。

4.3.2 测试过程

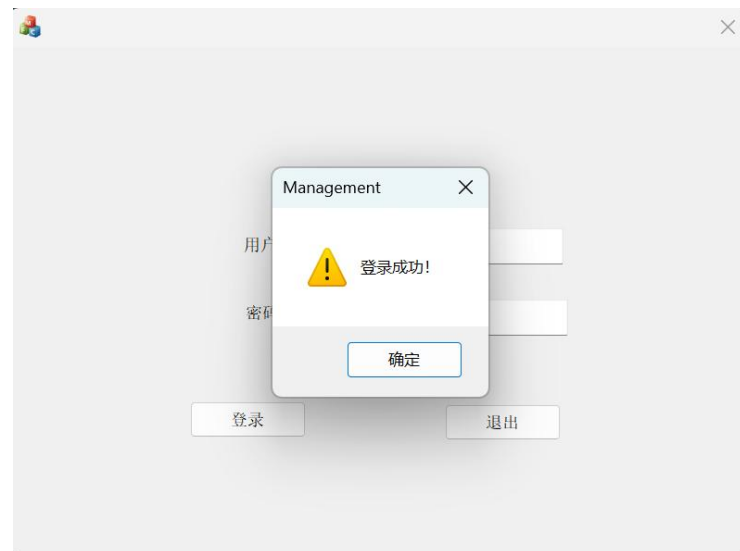
1. 启动系统，进入登录界面；



2. 输入数据库中已存在的用户账号与密码（如学生、教师或管理员账号）；



3. 点击登录按钮提交登录请求。



4.3.3 测试结果

系统能够正确校验用户账号信息，并根据用户角色类型进入对应功能界面：学生用户进入学生端界面，教师用户进入教师端界面，管理员用户进入管理员端界面。不同角色之间功能互不干扰，权限边界清晰，测试结果符合设计预期。

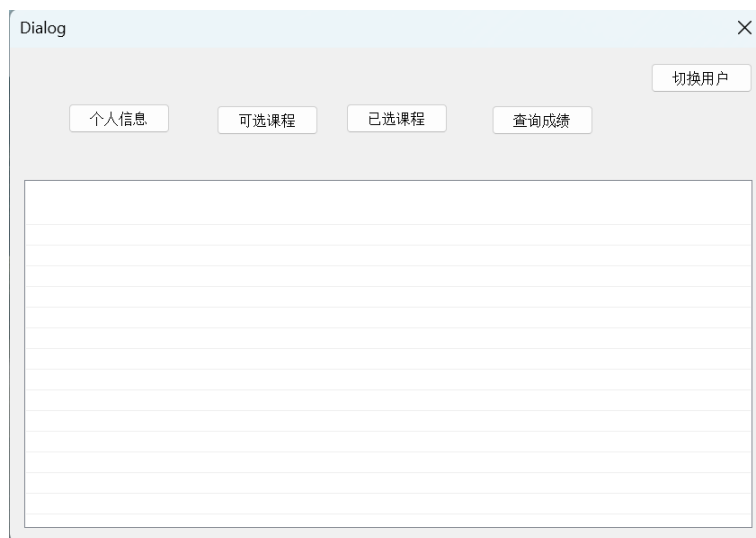
4.4 学生选课与成绩查询测试

4.4.1 测试目的

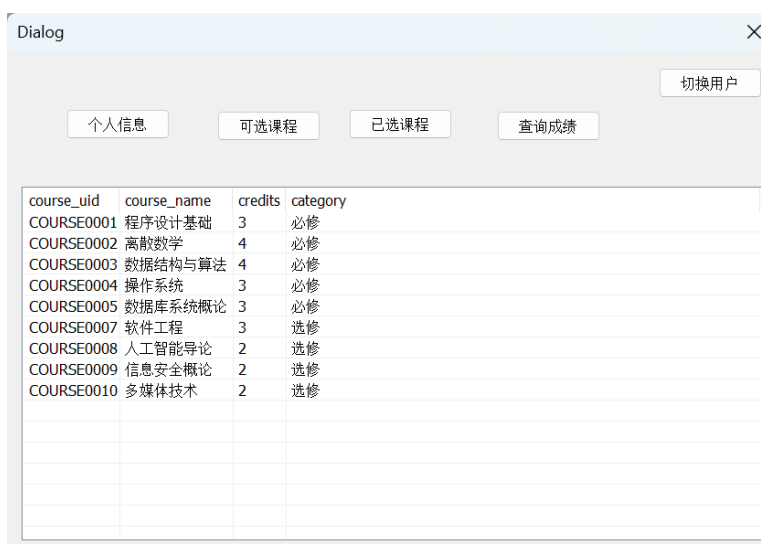
验证学生端选课功能与成绩查询功能是否正常，选课数据是否能够正确写入数据库。

4.4.2 测试过程

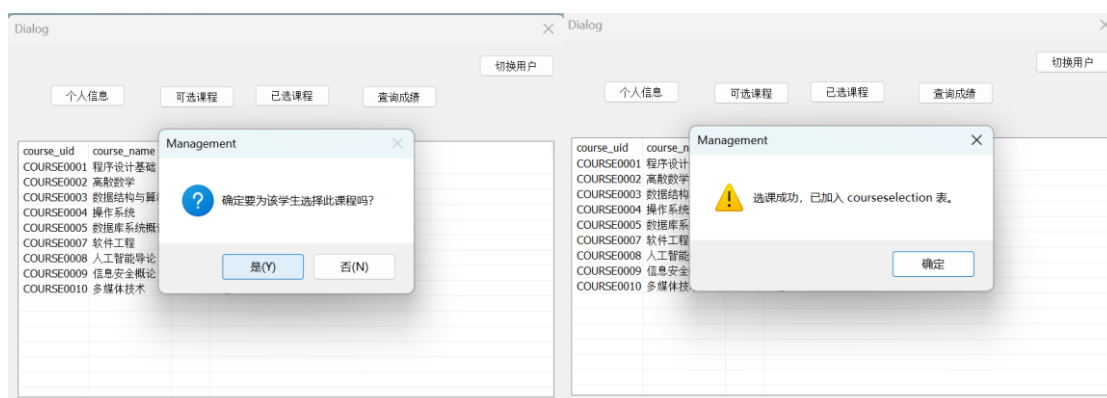
1. 以学生账号登录系统，进入学生端界面；

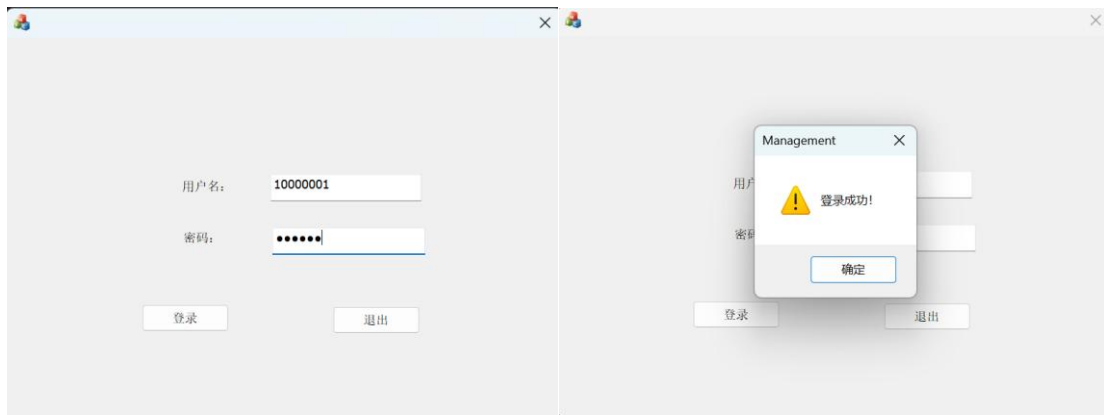


2. 查询系统中已开放的课程信息；

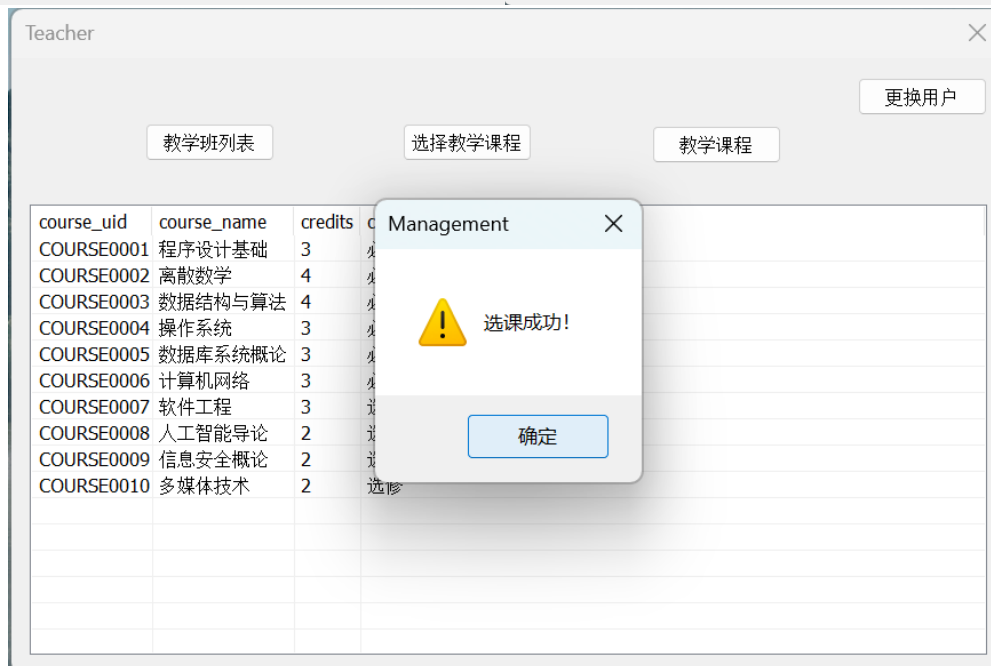
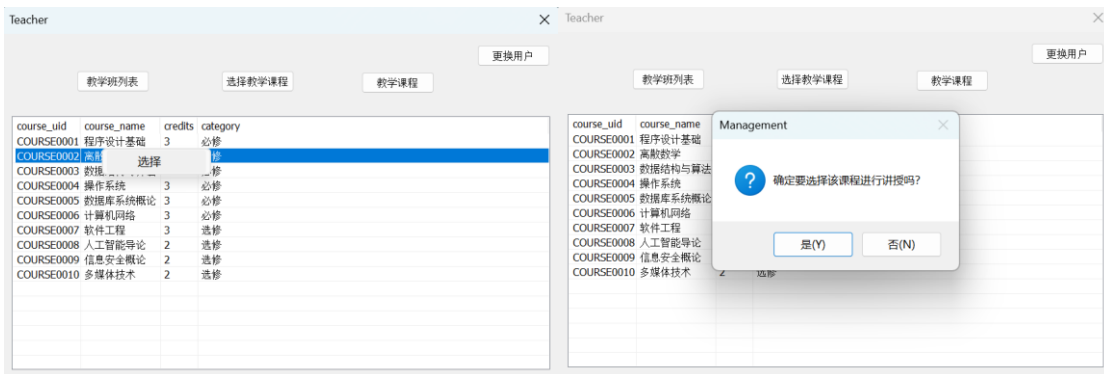


3. 选择课程并执行选课操作；

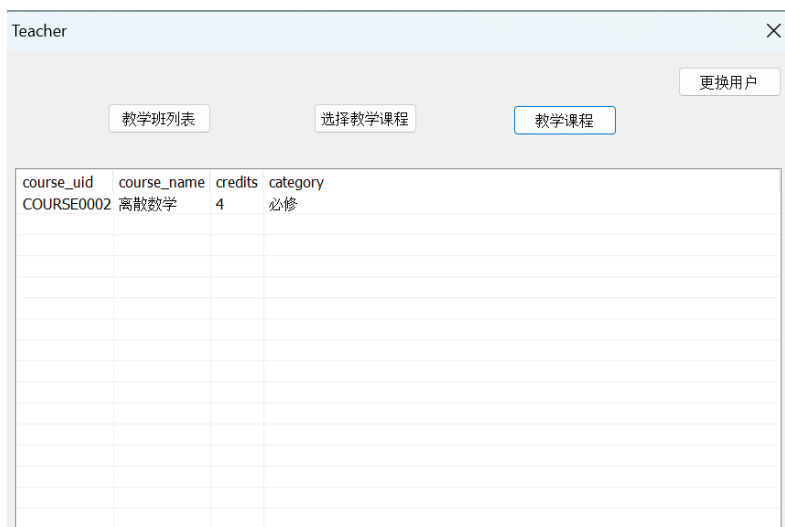




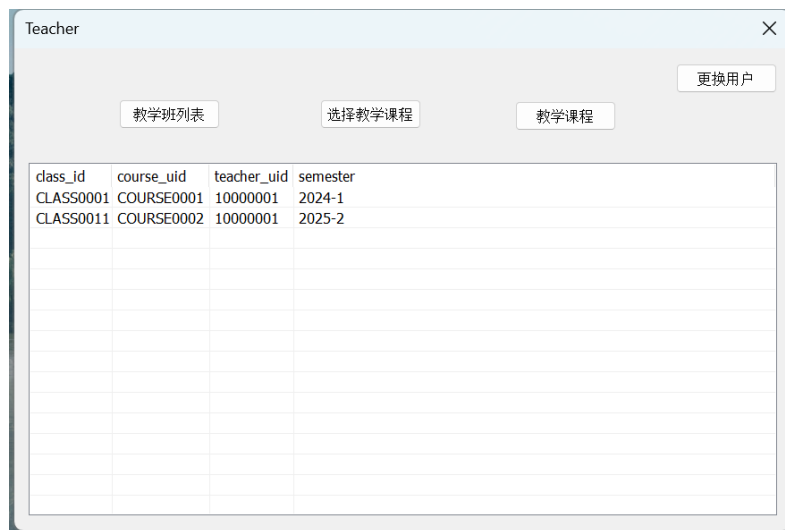
2. 在授课课程选择界面选择某一课程并确认授课;



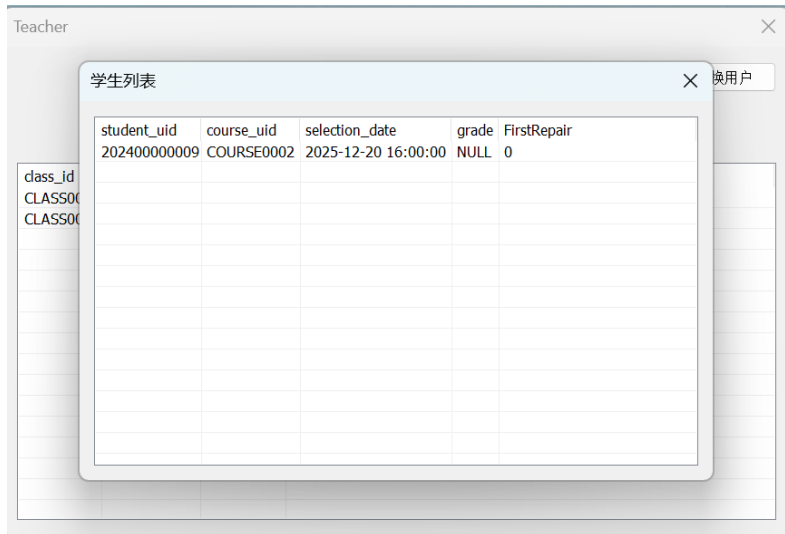
3. 查看数据库中授课记录表是否生成对应记录;



4. 查询教学班表，确认是否自动生成对应教学班；



5. 在教师端界面查看教学班信息。



4.5.3 测试结果

教师授课操作能够正确写入授课记录表，数据库触发器在授课记录插入后自动生成对应教学班。教学班信息在教师端界面中能够正确展示，且与授课课程及学期信息一致，验证了

系统在教学组织过程中的自动化与一致性。

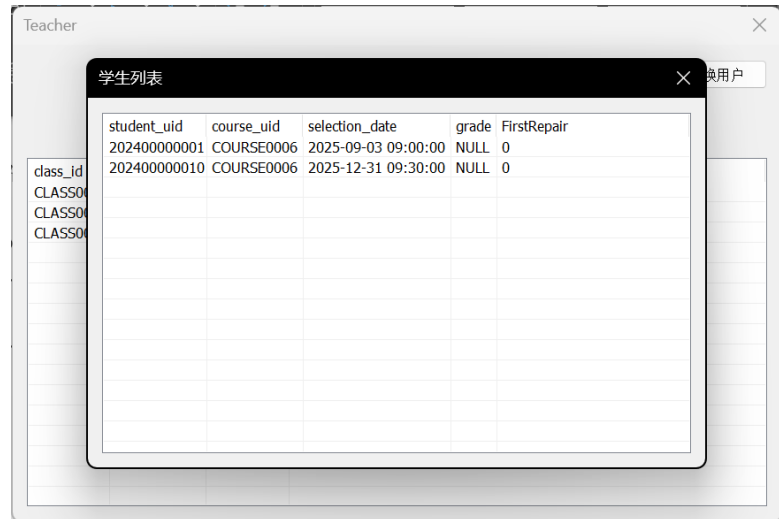
4.6 教学班学生列表与选课记录维护测试

4.6.1 测试目的

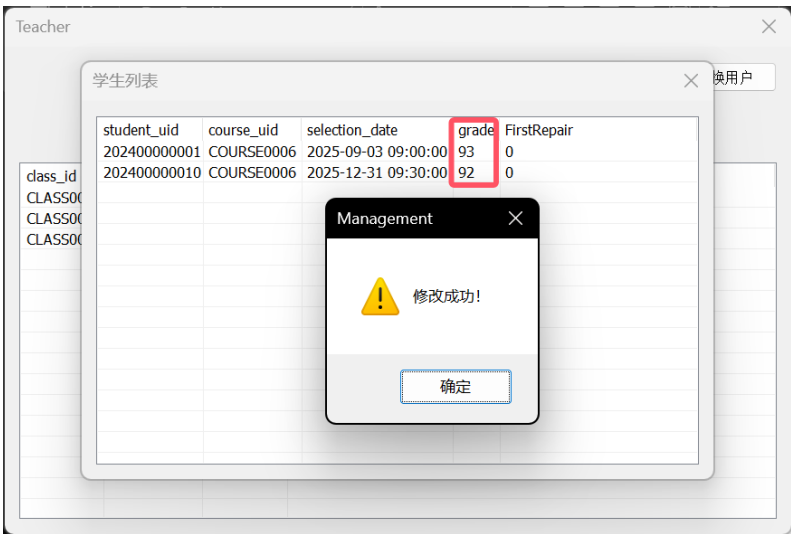
教师端教学班学生列表展示功能，以及选课记录编辑操作的正确性。

4.6.2 测试过程

1. 教师在教师端界面中选择某一教学班，打开教学班学生列表界面，查看学生选课记录；



2. 对选课记录中的部分字段进行编辑操作；



3. 提交修改后，检查数据库中对应记录是否更新。

student_uid	course_uid	selection_date	grade	FirstRepair
202400000001	COURSE0001	2024-03-01 09:00:00	85.50	0
202400000001	COURSE0001	2024-09-01 09:00:00	91.00	1
202400000001	COURSE0002	2025-12-15 00:00:00	NULL	1
202400000001	COURSE0003	2024-03-02 09:30:00	78.00	0
202400000001	COURSE0006	2025-09-03 09:00:00	93.00	0
202400000002	COURSE0001	2024-09-02 09:00:00	NULL	0
202400000002	COURSE0002	2024-03-01 10:00:00	88.00	0
202400000002	COURSE0002	2025-12-15 00:00:00	NULL	1
202400000002	COURSE0003	2025-09-15 10:30:00	NULL	0
202400000002	COURSE0005	2024-03-03 11:00:00	92.00	0
202400000003	COURSE0001	2025-10-02 14:00:00	NULL	0
202400000003	COURSE0003	2024-03-02 14:00:00	69.50	1
202400000003	COURSE0004	2024-03-04 09:00:00	NULL	0
202400000004	COURSE0004	2025-10-20 08:30:00	NULL	0
202400000004	COURSE0008	2024-03-05 13:00:00	95.00	0
202400000005	COURSE0001	2025-11-05 13:00:00	NULL	0
202400000005	COURSE0009	2024-03-06 15:00:00	73.00	0
202400000006	COURSE0003	2025-11-18 15:30:00	NULL	0
202400000006	COURSE0006	2024-03-01 08:30:00	81.75	0
202400000007	COURSE0005	2025-12-01 09:00:00	NULL	0
202400000007	COURSE0007	2024-03-02 16:00:00	87.00	0
202400000008	COURSE0002	2024-03-03 09:30:00	60.00	1
202400000008	COURSE0008	2025-12-15 10:15:00	NULL	0
202400000009	COURSE0002	2025-12-20 16:00:00	NULL	0
202400000009	COURSE0010	2024-03-04 10:30:00	78.25	0
202400000010	COURSE0005	2024-03-05 11:30:00	NULL	0
202400000010	COURSE0006	2025-12-31 09:30:00	92.00	0

4.6.3 测试结果

系统能够正确展示教学班对应的学生选课记录，并支持对选课数据的可视化编辑。修改操作能够准确定位到唯一选课记录，数据库更新结果正确，验证了教学班学生管理功能的可用性与安全性。

4.7 管理员基础数据维护测试

4.7.1 测试目的

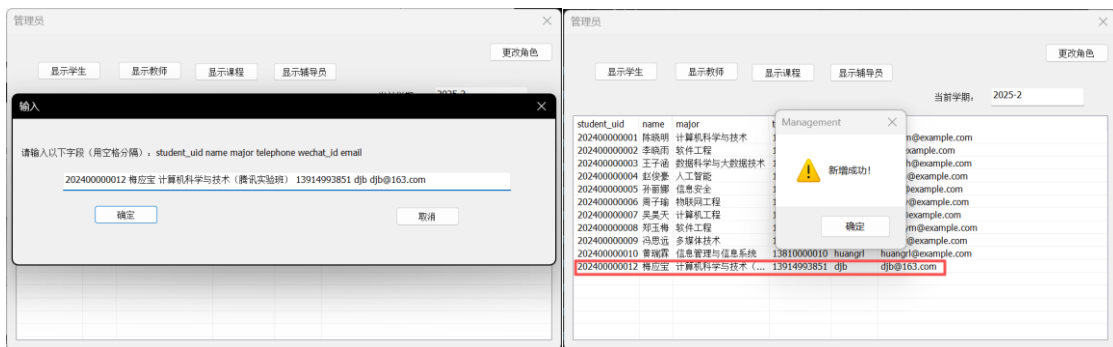
验证管理员端对基础数据的维护功能及数据库联动删除机制。

4.7.2 测试过程

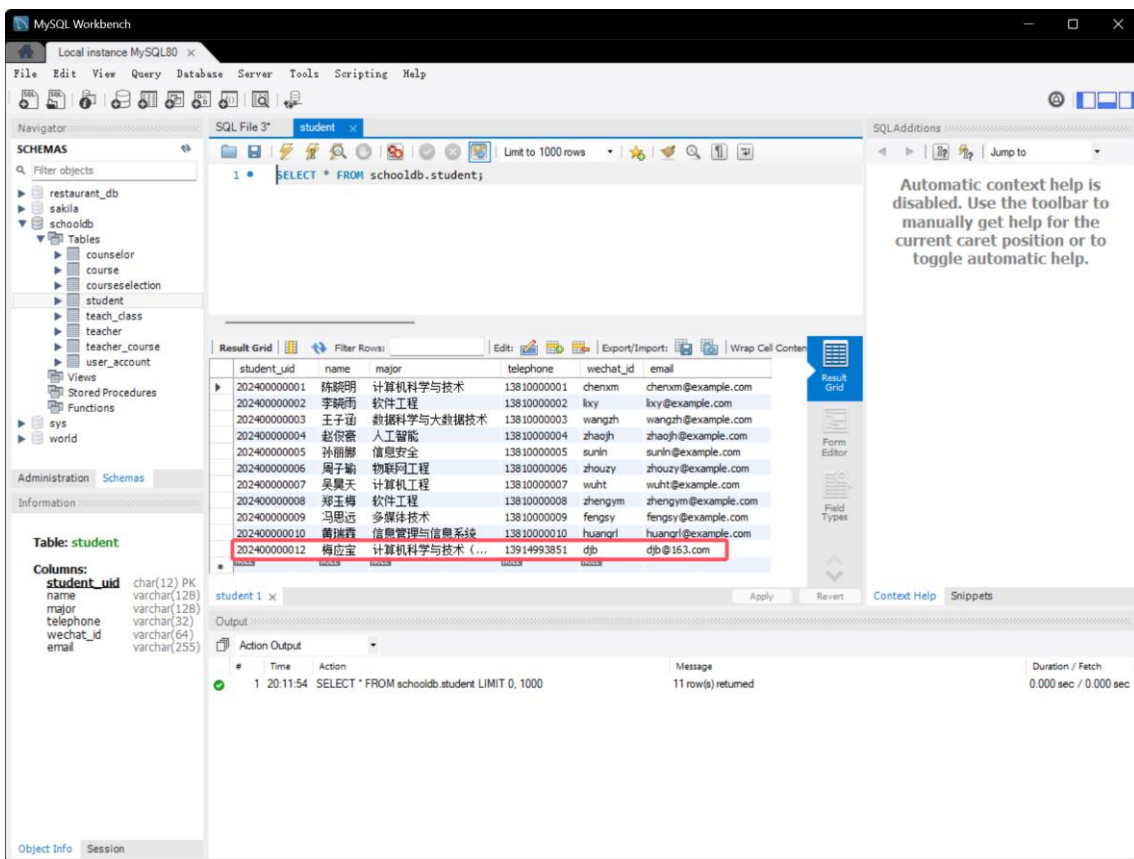
1. 以管理员账号登录系统；



2. 对学生、教师或课程等基础数据执行新增、修改与删除操作；



3. 在数据库中检查相关表数据变化情况。



4. 7. 3 测试结果

管理员端能够正确完成基础数据维护操作。当执行删除操作时，数据库触发器能够自动清理关联数据，未出现外键约束错误，保证了系统数据的一致性与完整性。

4. 8 选课—分班—成绩管理联通测试

4. 8. 1 测试目的

本测试以“学生选课—教务分班—教师录入成绩—学生查询成绩”为主线，验证系统在多角色协同场景下是否能够形成完整、连贯的数据流转闭环，重点检查选课数据、教学班数据及成绩数据在不同模块之间的一致性与正确性。

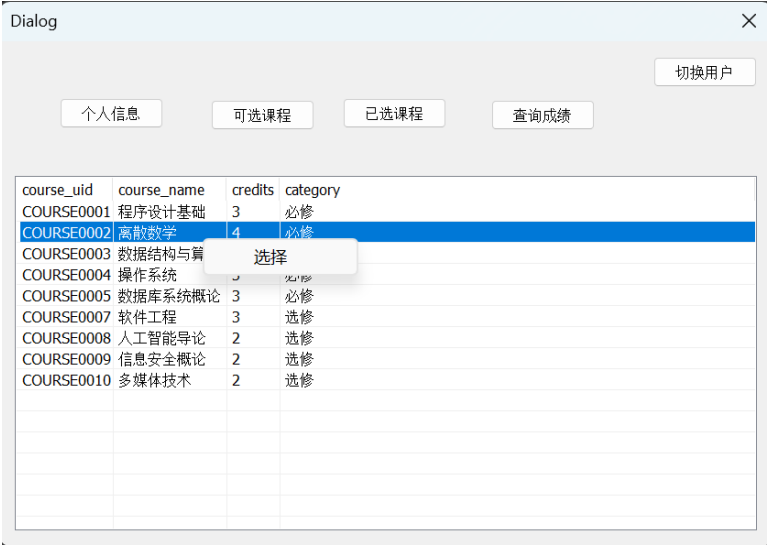
4. 8. 2 测试环境与初始条件

测试前，首先执行系统数据库初始化脚本，完成学生、教师、课程、教学班及用户账户等基础数据的创建与示例数据导入，确保数据库处于初始可测试状态。系统中各角色账号均已配置完成，可正常登录。

4.8.3 测试过程

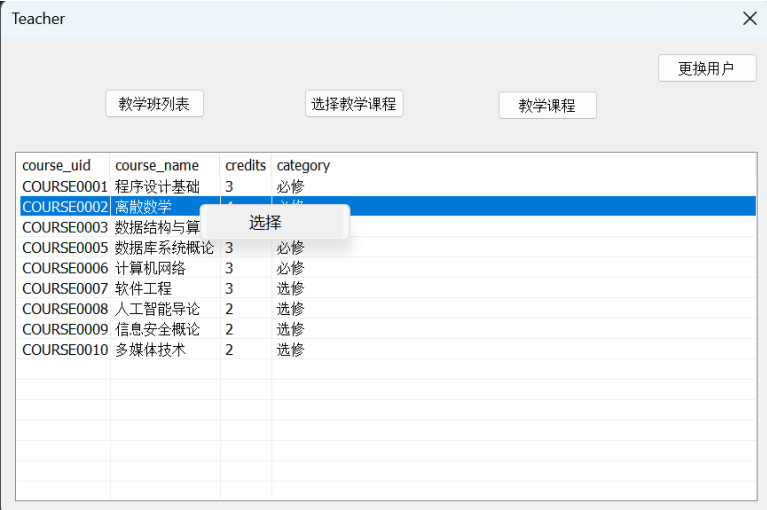
1. 学生选课阶段

以学生账号登录系统，进入学生端界面，选择指定学期内开放的课程并提交选课请求。系统提示选课成功，同时在数据库选课记录表中生成对应的选课数据，记录学生编号、课程编号、选课时间及修读状态等信息。



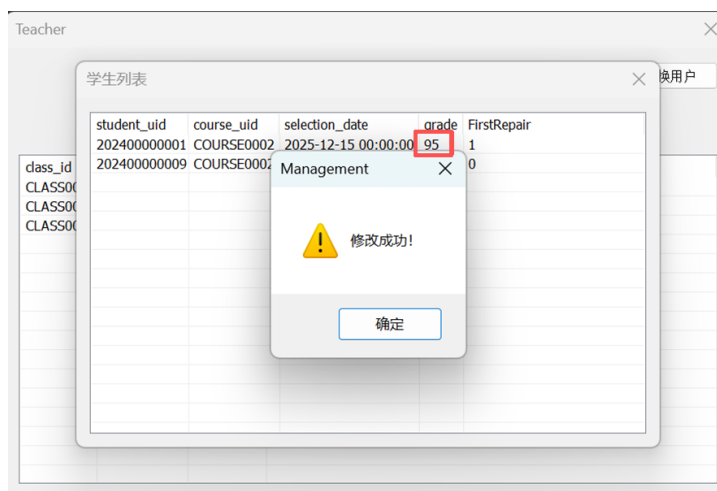
2. 教师选课阶段

以教师账号登录系统，进入教师端界面，选择指定学期内开放的课程并提交教授课请求。系统提示选课成功，系统根据课程授课情况及预设规则生成对应教学班，并将已选课学生关联到相应教学班中，完成从“选课记录”到“教学组织”的转换。



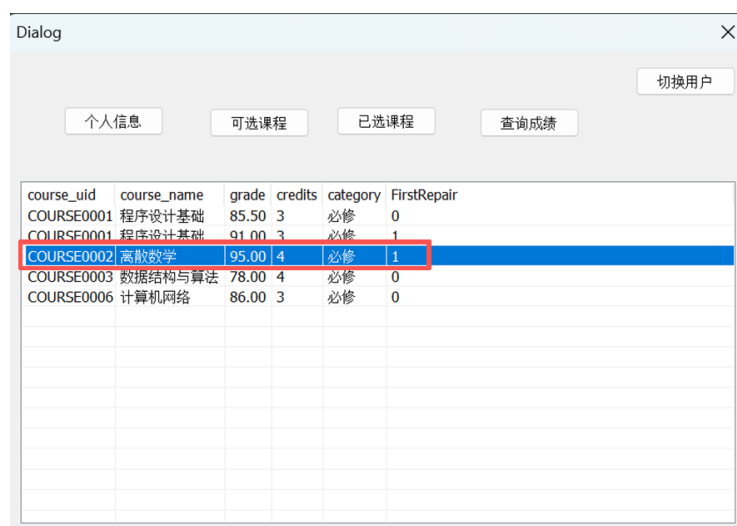
3. 教师录入成绩阶段

教师账号登录系统后，进入教师端界面，选择本人负责的教学班并查看学生名单。教学结束后，教师在系统中为教学班学生录入课程成绩，成绩信息写入数据库并与对应选课记录关联。



4. 学生查询成绩阶段

学生重新登录系统，进入成绩查询界面，查看所选课程的成绩信息。系统从数据库中读取成绩数据并在界面中展示。



4.8.4 测试结果

测试结果表明，系统能够正确支持该完整业务流程：

学生选课，数据库中成功生成选课记录；教务分班操作能够将选课学生正确组织到对应教学班；教师录入的成绩数据能够准确写入数据库；学生端查询到的成绩结果与数据库中存储的成绩记录保持一致。各阶段数据衔接顺畅，未出现数据缺失或不一致现象。

4.8.5 测试结论

通过该联通测试用例验证，系统在多角色协作场景下能够稳定运行，成功实现了从选课、教学组织到成绩管理的完整业务闭环，满足模拟教务管理系统在功能完整性与数据一致性方面的设计要求。

4.9 测试结果分析与总结

通过上述测试，系统各功能模块均能够按照设计要求正常运行。系统在登录鉴权、选课管理、授课管理、教学班生成及成绩管理等方面形成了完整、稳定的业务闭环。测试结果表明，系统功能实现正确，数据流转一致，能够满足模拟教务管理系统的设计目标。

5 总结与体会

通过本次综合实训项目的设计与实现，小组完成了一套基于 C++ 与 MySQL 的模拟教务管理系统。系统围绕“学生选课—教师授课—教学班组织—成绩管理”的核心业务流程展开，采用面向对象方法进行系统建模，并通过图形界面与数据库相结合的方式，实现了较为完整的教务业务闭环。

在系统设计阶段，小组对教务管理的实际业务流程进行了分析，将系统功能按角色划分为学生端、教师端和管理员端，并据此完成了模块划分与主要类设计。通过统一的登录与角色分流机制，不同用户只能访问与其权限相符的功能模块，保证了系统结构的清晰性与权限边界的明确性。各业务界面类职责相对单一，围绕自身业务流程组织代码，避免了功能混杂。

在程序实现过程中，小组逐步体会到合理分层与代码复用的重要性。通过引入数据库访问辅助类，对数据库连接与 SQL 执行操作进行集中封装，降低了界面逻辑与数据访问逻辑之间的耦合度。同时，通用输入对话框类的引入，使用户输入采集与提示逻辑得到统一管理，减少了重复代码，提高了系统的可维护性。

在数据库设计方面，系统围绕选课与教学组织构建了相对完整的数据模型，并通过触发器机制实现关键业务规则的自动化处理，例如教师授课后自动生成教学班、基础数据删除时联动清理关联记录等。这种将部分业务规则下沉到数据库层的设计方式，有效保证了数据一致性，降低了业务层遗漏处理逻辑的风险。

5.1 系统开发过程中遇到的问题及解决方法

在系统开发过程中，小组也遇到了一些具有代表性的问题，并在不断调试与改进中逐步解决。

例如，在早期实现阶段，界面类中直接编写了较多数据库操作代码，导致程序结构不够清晰。针对这一问题，小组对代码结构进行了调整，将数据库访问操作集中封装到数据库访问辅助类中，使界面类专注于交互与流程控制，从而改善了系统的整体结构。

此外，在选课、授课与教学班之间的数据关联处理中，若仅在程序层维护这些关系，容易出现遗漏或数据不一致的问题。为此，小组在数据库层引入触发器机制，将关键业务规则交由数据库自动执行，从而提高了系统的稳定性与一致性。

在教师端和管理员端的数据维护过程中，数据输入与编辑的交互实现也存在一定难度。通过设计通用输入对话框类，对输入流程进行统一封装，使界面交互更加规范，也降低了输入错误的发生概率。

在系统开发过程中，小组在教师端窗口的界面交互设计中遇到了多列表控件（ListCtrl）交互行为不一致的问题。教师端界面中根据不同业务需求展示了多张数据表，有的列表仅用于信息浏览，而有的列表需要支持右键操作或数据编辑。在初期实现时，为了提高代码复用性，对不同 ListCtrl 控件统一绑定了右键菜单和相关消息处理逻辑，导致部分仅用于展示数据的列表在右键点击时仍弹出操作菜单，容易引起用户误解，影响界面交互的清晰性。针对这一问题，小组在后续开发中重新梳理了各列表控件的功能定位，根据是否允许用户进行数

据修改对列表控件进行区分处理，仅对具备编辑权限的列表启用右键操作和编辑功能，而对仅用于显示的列表屏蔽相关交互。通过这一调整，教师端不同列表在交互行为上实现了差异化控制，界面操作逻辑更加清晰，也使小组进一步认识到在界面程序设计中需要将业务语义与交互设计相结合，从而提升系统的易用性与专业性。

5.2 系统的优点分析

通过本次实训系统的设计与实现，小组总结出系统具有以下几个方面的优点：

首先，系统业务流程完整，覆盖了教务管理中“选课—教学组织—成绩管理”的核心环节，并通过多角色协同实现了真实教务场景的模拟，整体功能结构清晰。

其次，系统在结构设计上采用了较为清晰的分层思想。界面控制类、数据访问类以及辅助类职责明确，有效降低了模块之间的耦合度，便于理解和维护，体现了面向对象设计中封装与单一职责原则的应用。

再次，数据库设计合理，通过主键、外键及触发器机制保证了数据的一致性与完整性。将部分业务规则交由数据库自动处理，减少了程序层的复杂度，提高了系统在多角色操作场景下的稳定性。

此外，系统在界面交互方面具备一定的通用性设计，例如通用输入对话框和可视化表格编辑方式，提高了数据维护效率，也增强了系统的可操作性。

5.3 系统的不足与改进方向

尽管系统已基本实现设计目标，但仍存在一些不足之处，有待在后续工作中进一步完善。

首先，系统目前主要针对单用户、低并发场景进行设计，对于高并发选课、冲突检测等复杂情况支持有限，尚未引入并发控制与事务级调度机制。

其次，部分业务逻辑仍由界面类直接触发完成，尚未进一步抽象为独立的业务服务层。在系统规模扩展或业务规则复杂化的情况下，代码结构仍有进一步优化空间。

此外，系统对教务规则的支持仍较为基础，例如课程时间冲突检测、成绩统计分析等功能尚未实现，需要在后续版本中逐步补充。

最后，系统界面交互形式较为简单，对异常情况的提示和引导仍可进一步完善，以提升系统的易用性和用户体验。

参考文献：

- [1] Microsoft. *MFC 桌面应用程序开发指南* [EB/OL].
- [2] Oracle Corporation. *MySQL C API Developer Guide* [EB/OL].
- [3] Bjarne Stroustrup. *The C++ Programming Language* (4th Edition). Addison-Wesley, 2013.

小组编号	*班第*组	组长	202383290121 梅应宝
学号	姓名	具体任务（分工）	
202383290121	梅应宝	系统框架搭建，数据库搭建，登录窗口实现，管理员窗口实现	
202383290382	董建标	系统框架搭建，教师窗口实现	
202383290102	丁诚诚	系统框架搭建，学生窗口实现	

Github 仓库: [MEIYBAO/OOD-Project: 面向对象程序设计课程设计](#)